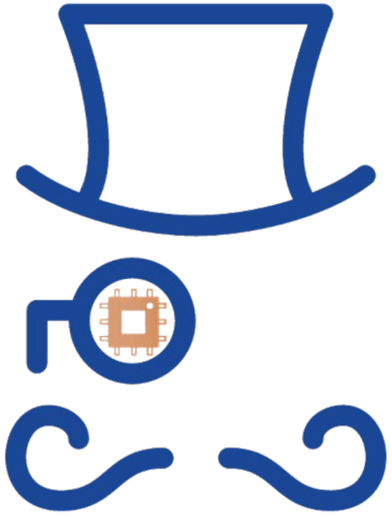
Pablo COLIN - Léa TEXIER

Custom Instruction RISCV

Summary

- Context
- Problem
- Software
- Hardware
- Performances
- Conclusion

Context - Department of Secure Systems and Architectures (SAS)

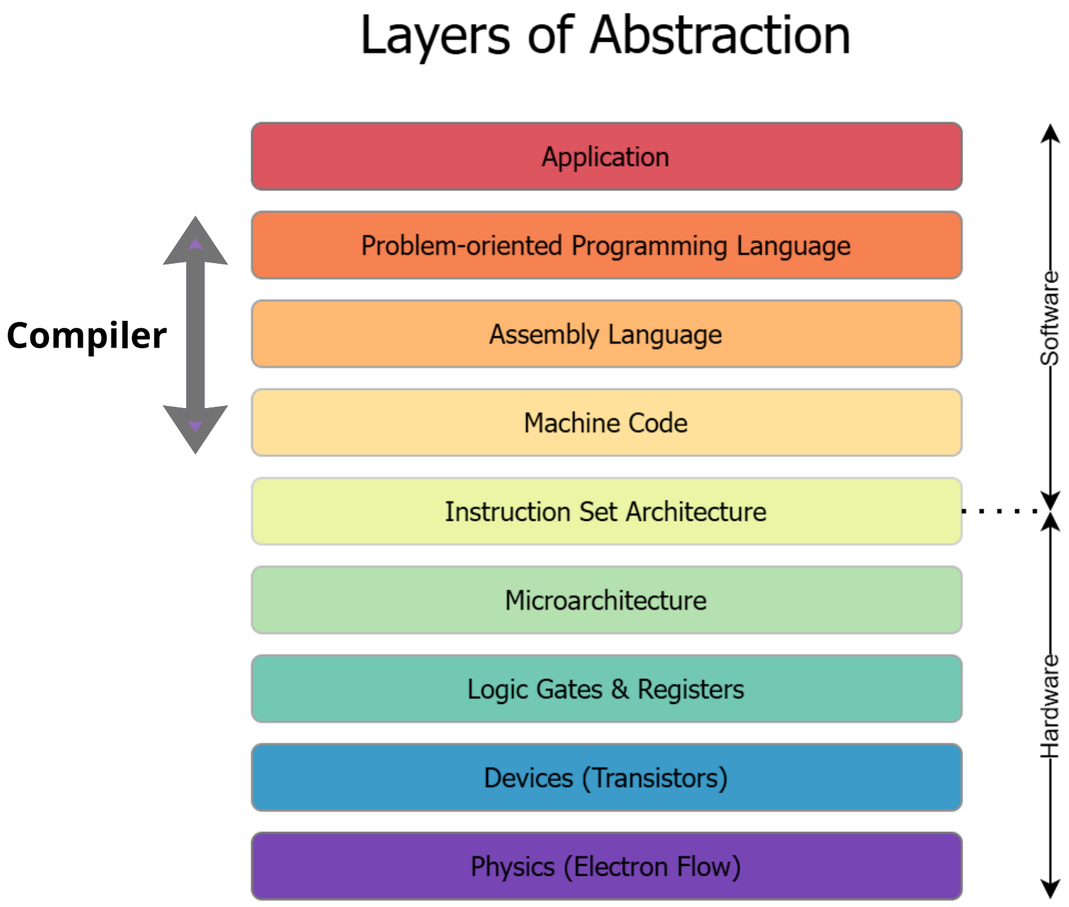


PEPR ARSENE

- System security: Cryptographic key
- Hardware attacks: Laser, Electromagnetic
- Test bench: Cybersecurity IP, **Custom instruction**

Context - Layers of abstraction

- **Application** : What we see = button, apps
- **Problem-oriented Programming Language** : Python, C++, Java
- **Assembly Language** : readable machine code
- **Machine code** : binary file (0 and 1)
- **Instruction Set Architecture** : Understandable by the processor
- **Hardware** : our Ibex



Context - The compiler

```
int main() {
        int a = 15;
        int b = 4;
        int c;
        c = a + b;

        return 0;
}
```

C file

Binary file

The compiler converts **human language** into **computer language**.
It is like a **translator** human/machine.
It allows us to write code easily.
Compile the entire program into an executable file.
He can also make a few **improvements**.

Problem - Standard Processor

Traditional processors execute standard instructions.
As a result, some tasks run **slowly**.
Or very **resource-intensive and computationally expensive.**

↓

Standard processor architectures are **not optimized** for certain specific tasks.

↓

Add custom instructions.
Execution in a **single instruction**.
Improves performance and **reduces the number of clock cycles.**

Problem - Solution: RISC-V

- It is an **ISA** (Instruction Set Architecture) : includes all instructions, registers, and rules.
- RISC-V is **extensible** : we can add custom instructions
- These instructions are integrated into the processor design (IBEX).
- The processor is modified to support and execute the new custom instructions.

inst[4:2] / inst[6:5]	000	001	010	011	100	101	110	111 (> 32b)
00	LOAD	LOAD-FP	*custom-0*	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	*custom-1*	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	*reserved*	*custom-2/rv128*	48b
11	BRANCH	JALR	*reserved*	JAL	SYSTEM	*reserved*	*custom-3/rv128*	≥ 80b

Table 8.1: RISC-V base opcode map, inst[1:0]=11

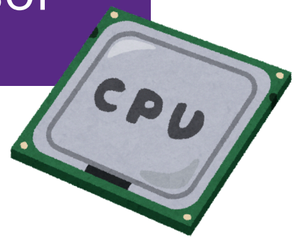
Software - What is an Instruction on a Processor

Programmable language

Instruction Set Architecture (ISA)

Toolchain

C file containing assembly language lines → *Compiler* → Binary file → Processor




```c
int main() {
        int a,b,c,s;

        asm volatile
        ("addi %0, x0, 0x5"
        : "=r" (a));
        asm volatile
        ("addi %0, x0, 0x3"
        : "=r" (b));
```

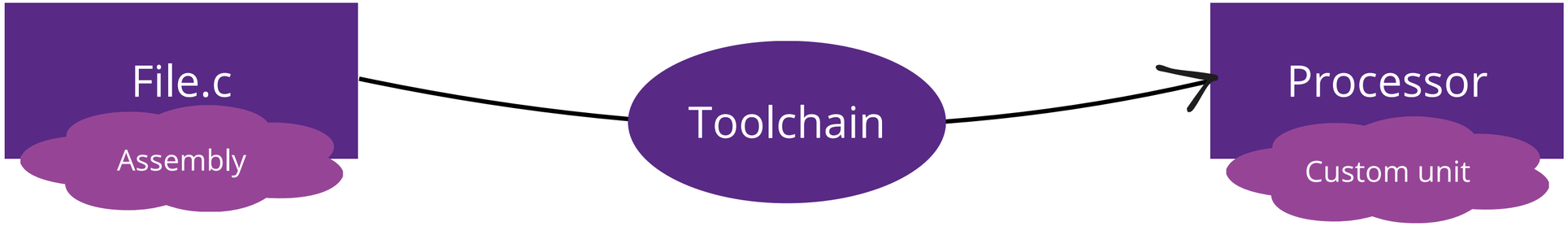
```
li      a5,5
li      a4,3
mod     a5,a5,a4
lui     a4,0x20
sw      a5,0(a4)
lui     a5,0x20
li      a4,1
sw      a4,8(a5)
li      a0,0
```

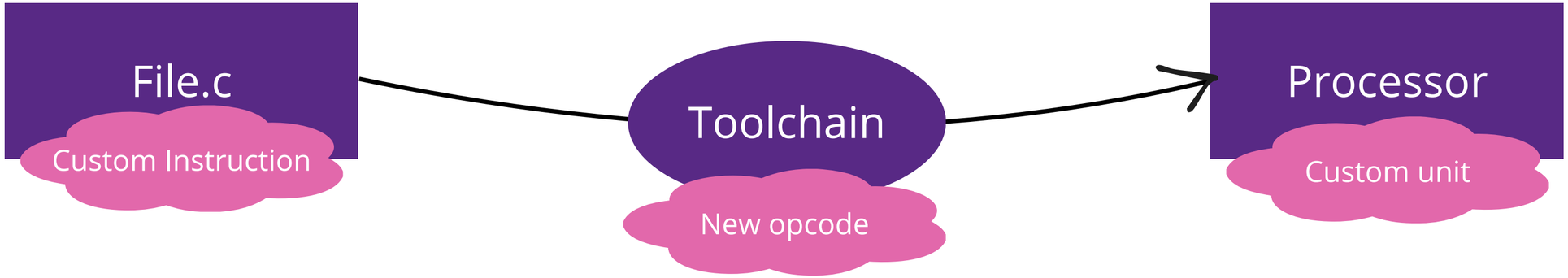
Code C with assembly

```
@00100084
6F 00 80 08 6F 00
06 82 86 82 06 83
06 86 86 86 06 87
06 8A 86 8A 06 8B
06 8E 86 8E 06 8F
17 0D 00 00 13 0D
63 57 BD 01 23 20
81 45 EF 00 40 02
62 00 73 00 50 10
```

(Hex file)

Software - Our improvement





Software - Custom Instuctions

- **rd** = Destination register
- **rs** (1 or 2) = Source register
- **imm** (12 or 20) = Immediate Value of 12 or 20 bits

mod rd rs1 rs2 rd = rs1 mod rs2

b = 5, c = 3 → mod a b c → a = b % c → a = 2

mroli rd rs1 imm12 rd = right_rotation(rs1, imm12)

b = 0x523 → mroli a b 7 → a = b >> 7 → a = 0x4600000A

mdroli rd imm20 rd = right_rotation(imm20, 7)

a = 0 → mdroli a 0x523 → a = b >> 7 → a = 0x4600A

jumb rs1 rs2 bimm12 if (rs1 = rs2) then PC ← PC + 4 else PC ← PC + 2

a = 3, b = 3, c = 4 → jumb a b label , mod c a b , mod a c b → a = c mod b → a = 1, b = 3, c = 4

Software - Custom Instuctions

mod - type R → *Opcode cust0*
mroli - type I → *Opcode cust1*
mdroli - type U → *Opcode cust2*
jumb - type B → *Opcode cust3*

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

	31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
mod	funct7	rs2	rs1	funct3	rd	opcode
Mask	1111111	00000	00000	111	00000	1111111
Match	0000001	00000	00000	000	00000	0001011
mroli	imm[11:0]		rs1	funct3	rd	opcode
Mask	000000000000		00000	111	00000	1111111
Match	000000000000		00000	001	00000	0101011
mdroli	imm[31:12]				rd	opcode
Mask	00000000000000000000				00000	1111111
Match	00000000000000000000				00001	1011011
jumb	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
Mask	0000000	00000	00000	111	00000	1111111
Match	0000000	00001	00001	001	00000	1111011

Software Modification

Software - Opcode/Mask

Riscv-opcodes Repositery [1]

riscv-gnu-toolchain/riscv-opcodes/extension/rv_i

```
# rv_i
mod     rd rs1 rs2    31..25=0    14..12=0 6..2=2    1..0=3
mroli   rd rs1 imm12              14..12=0 6..2=0xA  1..0=3
mdroli  rd imm20                            6..2=0x16 1..0=3
jumb    bimm12hi rs1 rs2 bimm12lo 14..12=0 6..2=0x1E 1..0=3
```

riscv-gnu-toolchain/riscv-opcodes/encoding.out.h

```
DECLARE_INSN(mod, MATCH_MOD, MASK_MOD)
DECLARE_INSN(mroli, MATCH_MROLI, MASK_MROLI)
DECLARE_INSN(mdroli, MATCH_MDROLI, MASK_MDROLI)
DECLARE_INSN(jumb, MATCH_JUMB, MASK_JUMB)
```

Opcodes/Masks for our custom functions

```
#define MATCH_MOD 0xb
#define MASK_MOD 0xfe00707f
#define MATCH_MROLI 0x2b
#define MASK_MROLI 0x707f
#define MATCH_MDROLI 0x5b
#define MASK_MDROLI 0x7f
#define MATCH_JUMB 0x7b
#define MASK_JUMB 0x707f
```

Software - Declaration

RISC-V GNU Compiler Toolchain Repositery [2]

Declaration

riscv-gnu-toolchain/binutils/include/opcode/riscv-opc.h

```
#ifndef RISCV_ENCODING_H
#define RISCV_ENCODING_H
/* Custom Instuctions */
#define MATCH_MOD 0xb
#define MASK_MOD 0xfe00707f
#define MATCH_MROLI 0x2b
#define MASK_MROLI 0x707f
#define MATCH_MDROLI 0x5b
#define MASK_MDROLI 0x7f
#define MATCH_JUMB 0x7b
#define MASK_JUMB 0x707f

#endif /* RISCV_ENCODING_H */
#ifdef DECLARE_INSN

DECLARE_INSN(mod, MATCH_MOD, MASK_MOD)
DECLARE_INSN(mroli, MATCH_MROLI, MASK_MROLI)
DECLARE_INSN(mdroli, MATCH_MDROLI, MASK_MDROLI)
DECLARE_INSN(jumb, MATCH_JUMB, MASK_JUMB)
```

Definition

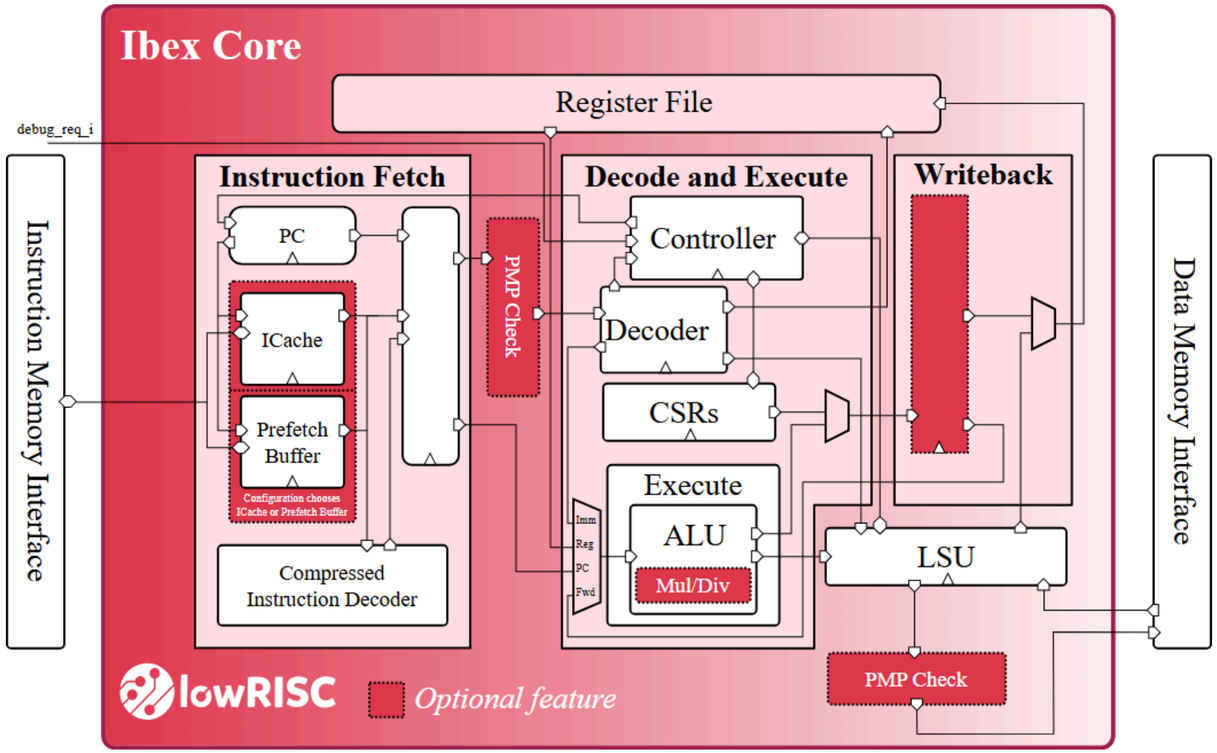
riscv-gnu-toolchain/binutils/opcodes/riscv-opc.c

```
const struct riscv_opcode riscv_opcodes[] =
{
/* name, xlen, isa, operands, match, mask, match_func, pinfo.  */

/* Custom Instructions */
{"mod",     0, INSN_CLASS_I, "d,s,t",   MATCH_MOD,  MASK_MOD,   match_opcode, 0},
{"mroli",   0, INSN_CLASS_I, "d,s,j",   MATCH_MROLI,    MASK_MROLI,     match_opcode, 0},
{"mdroli",  0, INSN_CLASS_I, "d,u",     MATCH_MDROLI,   MASK_MDROLI,    match_opcode, 0},
{"jumb",    0, INSN_CLASS_I, "s,t,p",   MATCH_JUMB,     MASK_JUMB,  match_opcode, INSN_CONDBRANCH},
```

Hardware Modification

Tools

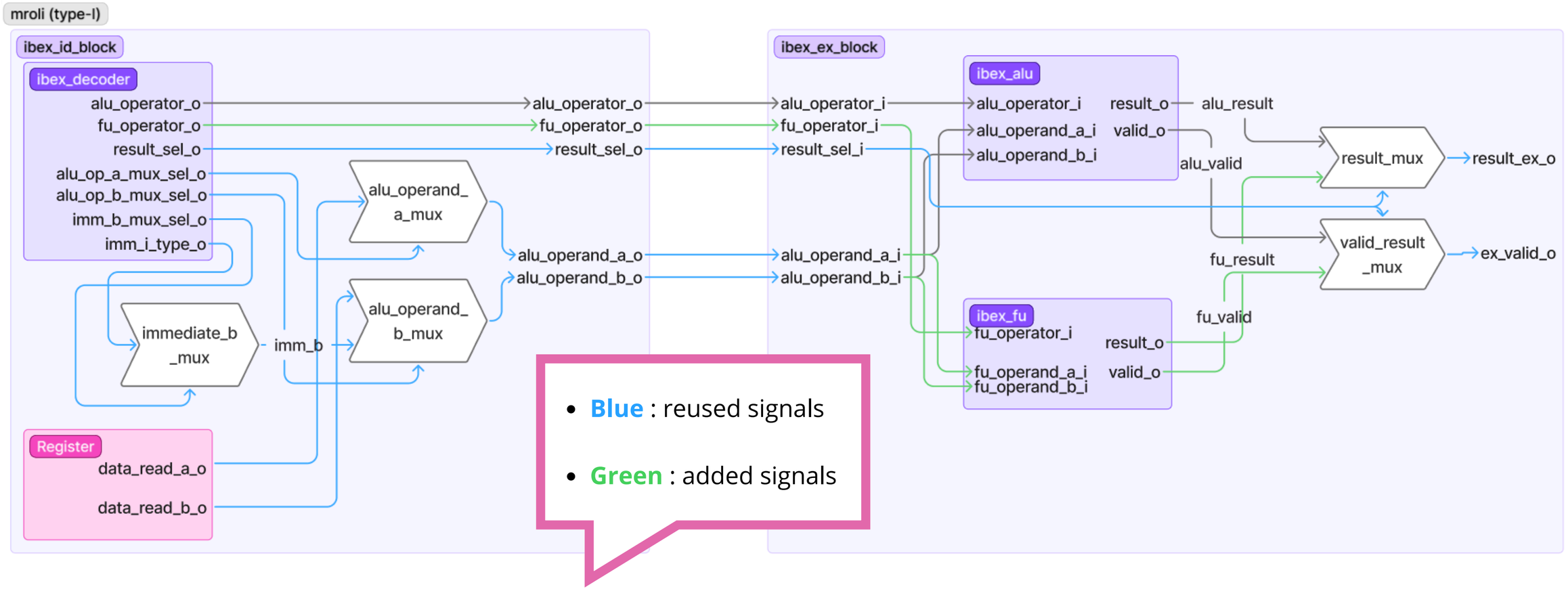


Processor = IBEX Risc-V

- **2** or 3 stages processor
- 32-bit RISC-V CPU core written in SystemVerilog
- **accepts customs instructions**
- supports lots of operations (I/E, M, C, B)

Hardware - MROLI example

Ibex RISC-V Core Repositery [3]



- **Blue** : reused signals
- **Green** : added signals

Hardware - Decoder for MROLI

```
OPCODE_CUSTOM1: begin
  rf_ren_a_o        = 1'b1;
  rf_we             = 1'b1;

  unique case (instr[14:12])
    3'b000:  begin
      illegal_insn = 1'b0;
    end
    default: begin
      illegal_insn = 1'b1;
    end
  endcase
end
```

```
OPCODE_CUSTOM1: begin
  alu_op_a_mux_sel_o  = OP_A_REG_A;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_b_mux_sel_o     = IMM_B_I;

  result_sel_o       = FU_RES;

  unique case (instr_alu[14:12])
    3'b000: fu_operator_o = FU_MROLI;
    default: ;
  endcase
end
```

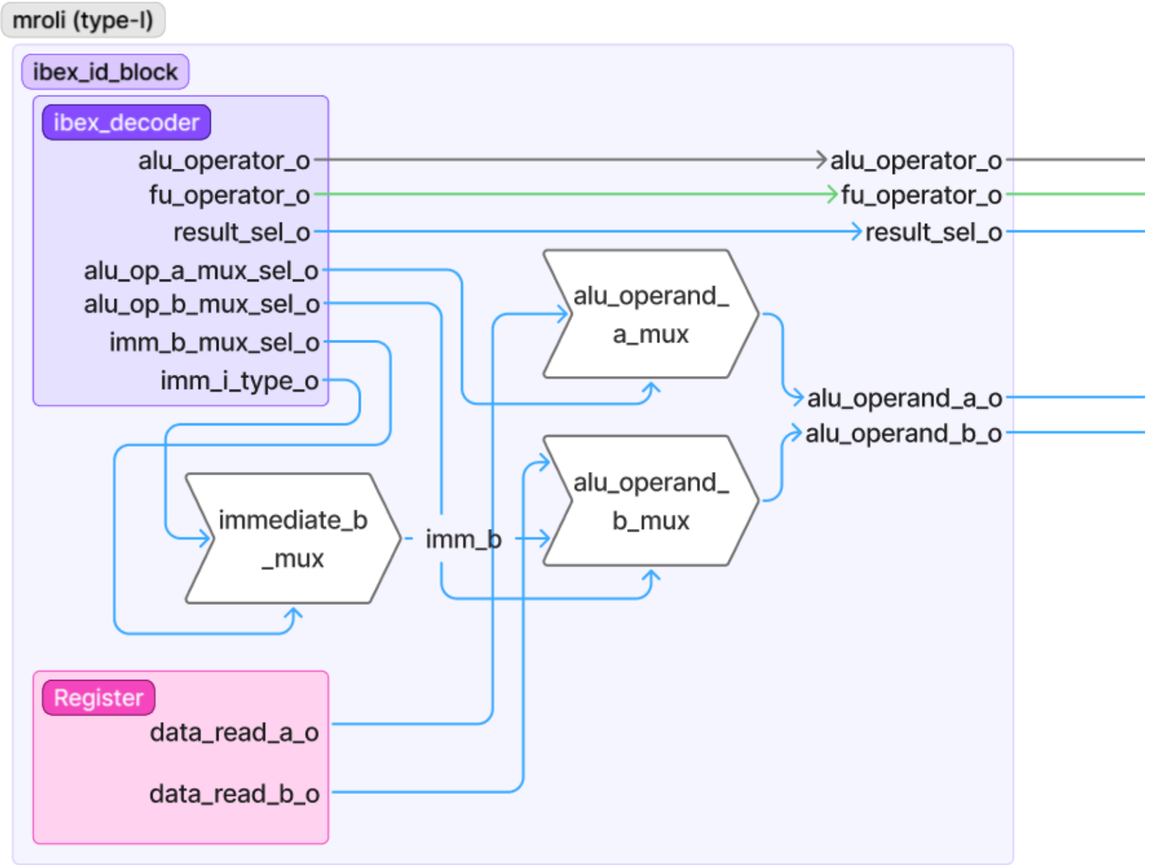
Hardware - Decoder for MROLI

mroli rd rs1 imm12

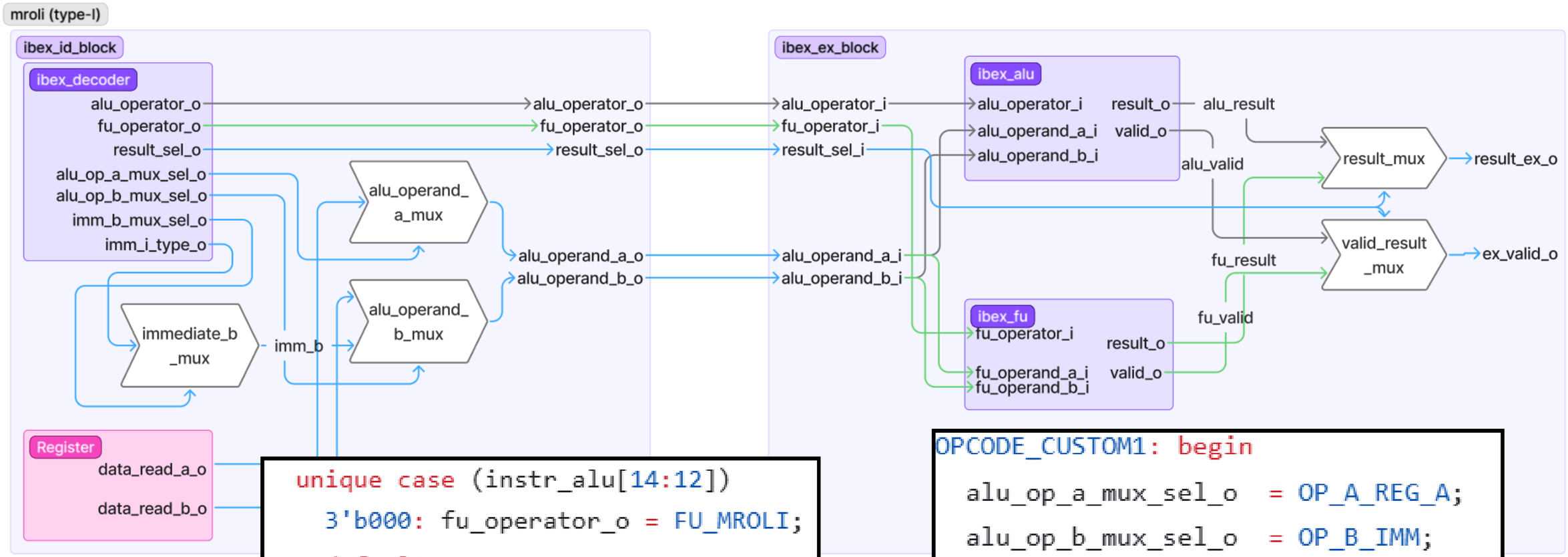
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
mroli	imm[11:0]												rs1					funct3 000			rd					opcode 0101011						

```
OPCODE_CUSTOM1: begin
  rf_ren_a_o        = 1'b1;
  rf_we             = 1'b1;

  unique case (instr[14:12])
    3'b000:  begin
      illegal_insn = 1'b0;
    end
    default: begin
      illegal_insn = 1'b1;
    end
  endcase
end
```



Hardware - Decoder for MROLI




```
unique case (instr_alu[14:12])
  3'b000: fu_operator_o = FU_MROLI;
  default: ;
endcase
end
```

```
OPCODE_CUSTOM1: begin
  alu_op_a_mux_sel_o  = OP_A_REG_A;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_b_mux_sel_o     = IMM_B_I;

  result_sel_o        = FU_RES;
```

Performances

```
(ibex-env) parallels@ubuntu-gnu-linux-24-04-3:~/Documents/PR/RV32$ ./run_test_arx_nc.sh
Received $finish() from Verilog, shutting down simulation.

Simulation statistics
=====================
Executed cycles:  64
Wallclock time:   0 s
Simulation speed: inf cycles/s (inf kHz)

Performance Counters
====================
Cycles:               60
Instructions Retired: 51
```

Simulation of a rotation using basic instructions

60 cycles

```
(ibex-env) parallels@ubuntu-gnu-linux-24-04-3:~/Documents/PR/RV32$ ./run_test_arx.sh
- src/lowrisc_ibex_sim_shared_0/rtl/sim/simulator_ctrl.sv:93: Verilog $finish
Received $finish() from Verilog, shutting down simulation.

Simulation statistics
=====================
Executed cycles:  62
Wallclock time:   0.002 s
Simulation speed: 31000 cycles/s (31 kHz)

Performance Counters
====================
Cycles:               58
Instructions Retired: 49
```

Simulation of a rotation using a custom instruction

58 cycles

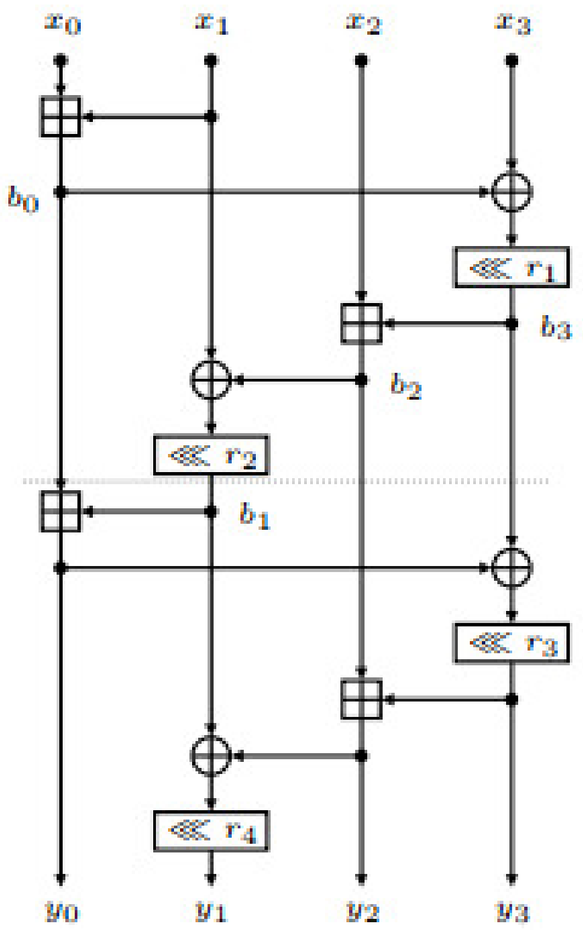
Conclusion

Summary :

- We can create any type of operation.
- We gained two cycles on the rotation.
- A simpler way to refer to custom instructions in the code

Next steps :

- Create secure instructions
- Concatenate an ARX in one instruction
- Create new types of instructions



The chacha quarter round

Appendix

[1] Git repositery: **Riscv-opcodes** Url: https://github.com/ riscv-collab/riscv-gnu-toolchain.git

[2] Git repositery: **RISC-V GNU Compiler Toolchain** Url: https://github.com/ riscv-collab/riscv-gnu-toolchain.git

[3] Git repositery: **Ibex RISC-V Core** Url: https://github.com/lowRISC/ibex

Thank you for your kind attention

Performances

Log File

```
TEST MOD
Test 1 (15 % 4): 3
Test 2 (Result1 % 4): 3
Test 3 (Result2 + 10): 13
Stress Test (chain): 0
Résultat final h: 5,
f: 5

TEST ROTI
Test 1 (15 roti 4): -268435456
Expected: -268435456
Test 2 (Result1 roti 4): 251658240
Expected: 251658240
Test 3 (Result2 + 10): 251658250
```

```
4501        li      a0,0
47f1        li      a5,28
02f4050b    mod     a0,s0,a5
02a7840b    mod     s0,a5,a0
02f4050b    mod     a0,s0,a5
02f4050b    mod     a0,s0,a5
001007b7    lui     a5,0x100
001006b7    lui     a3,0x100
```

Disassembly File

Trace File

```
4501    c.li    x10,0     x10=0x00000000
47f1    c.li    x15,28    x15=0x0000001c
02f4050b        INVALID
02a7840b        INVALID
02f4050b        INVALID
02f4050b        INVALID
001007b7        lui     x15,0x100       x15=0x00100000
001006b7        lui     x13,0x100       x13=0x00100000
77c78793        addi    x15,x15,1916    x15:0x00100000 x15=0x0010077c
```

Template